

Arrays in C

Data Structures & Memory Management

A Complete 10-Page Technical Reference

1. Logical Concept

An **array** in C is a collection of elements of the same data type stored in **contiguous (continuous)** memory locations.

Instead of declaring 5 separate variables for student marks, we declare one array variable that holds all 5 values.

Syntax:

```
data_type array_name[size];
```

Exam Definition: An array is a derived data structure that stores multiple elements of the same type in continuous memory locations under a single name.

2. Memory Representation

In memory, array elements are placed side-by-side. If the first element is at address 1000, and it's an `int` (4 bytes), the next is at 1004.



Zero-Based Indexing:

The first element is always accessed via index `0`. The last element is at `size - 1`.

3. Initialization & Access

There are several ways to initialize an array:

```
int arr[5] = {10, 20, 30, 40, 50}; // Full  
int arr[] = {1, 2, 3}; // Automatic size  
int arr[5] = {0}; // All elements zero
```

Accessing Elements:

```
printf("%d", arr[2]); // Accesses 3rd element
```

4. Basic Program: Array Sum

```
#include <stdio.h>
int main() {
    int marks[3] = {85, 90, 75};
    int sum = 0;
    for(int i=0; i<3; i++) {
        sum += marks[i];
    }
    printf("Total Marks: %d", sum);
    return 0;
}
```

5. Two-Dimensional Arrays

A 2D array is an "Array of Arrays", often visualized as a **Matrix** (Rows and Columns).

Syntax:

```
int matrix[rows][cols];
```

Example:

```
int a[2][2] = {{1, 2}, {3, 4}};
```

2D Array Implementation

Using nested loops to print a 2x2 matrix:

```
for(int i=0; i<2; i++) {  
    for(int j=0; j<2; j++) {  
        printf("%d ", a[i][j]);  
    }  
    printf("\n");  
}
```

6. Pros & Cons

Advantages	Disadvantages
Random access (Fast)	Fixed size (Static)
Less code (Loops)	Memory wastage if not filled
Easy data Sorting	Deletion/Insertion is slow

7. Key Points for Viva/Exams

- **Homogeneous:** Contains elements of ONLY one type.
- **Static:** Size cannot be changed during runtime.
- **Bounds:** C does not check array bounds (you can accidentally access `arr[10]` in an `arr[5]`).
- **Name:** The array name acts as a pointer to the first element.

Module Complete

You now understand Array structures, Memory, and Matrices in C Programming.